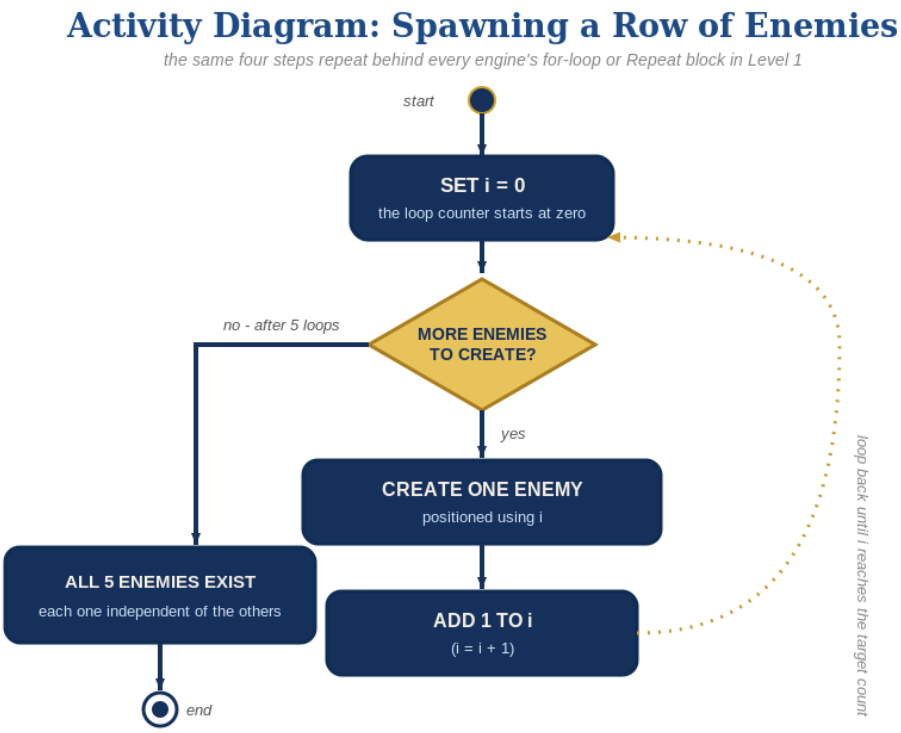
# Lesson 5 Teacher's Key

GameMaker

**INSTRUCTOR ONLY - do not distribute to students**

## Level 1 - Complete Worked Solution




*Every engine's Level 1 solution runs this same shape - a counter, a repeat check, one creation step, and an increment - even though the exact blocks or code syntax differ. Tracing a spawn bug means finding which of these four steps broke.*

*The spawn loop, in general - the same counter, repeat check, creation step, and increment behind Level 1 in every engine, whatever the exact blocks or syntax look like.*

A new obj_spawner with a Create event loop - obj_enemy and obj_bullet need zero changes from Lesson 4:

```
// obj_spawner's Create event:
for (var enemy_count = 0; enemy_count < 5; enemy_count++) {
    instance_create_layer(80 + enemy_count * 96, 80, "Instances",
obj_enemy);
}
```

## Level 2 - Complete Worked Solution

obj_bullet's Collision event with obj_enemy already reacts per pair of overlapping instances, so every instance the loop creates behaves independently and correctly with no extra code needed for Level 2.

## Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

### Sound: already works

audio_play_sound() in obj_bullet's Collision event runs independently for every enemy that gets hit - no changes needed.

### Visual: striped row

Set image_blend on each new instance right after creating it, using with(). make_color_rgb(200, 25, 55) is a deep crimson red and make_color_rgb(230, 100, 130) is a lighter dusty rose/pink - image_blend multiplies the sprite's own pixels by this tint, so alternating the two makes every other enemy a visibly different shade of red, not a blended new color. Written as a plain if/else rather than a one-line ternary - readable at a glance for a student who hasn't met the ternary operator yet (see this lesson's Book Notes entry: that's deliberately saved for a later, more advanced project):

```
for (var enemy_count = 0; enemy_count < 5; enemy_count++) {
    var new_enemy = instance_create_layer(80 + enemy_count * 96, 80,
"Instances", obj_enemy);
    with (new_enemy) {
        if (enemy_count % 2 == 0) {
            image_blend = make_color_rgb(200, 25, 55);
        } else {
            image_blend = make_color_rgb(230, 100, 130);
        }
    }
}
```

### Structure: spawner vs. behavior stay separate

obj_spawner's Create event only ever creates and positions enemies - collision-response and hit-sound logic stay entirely inside obj_enemy and obj_bullet's own events.

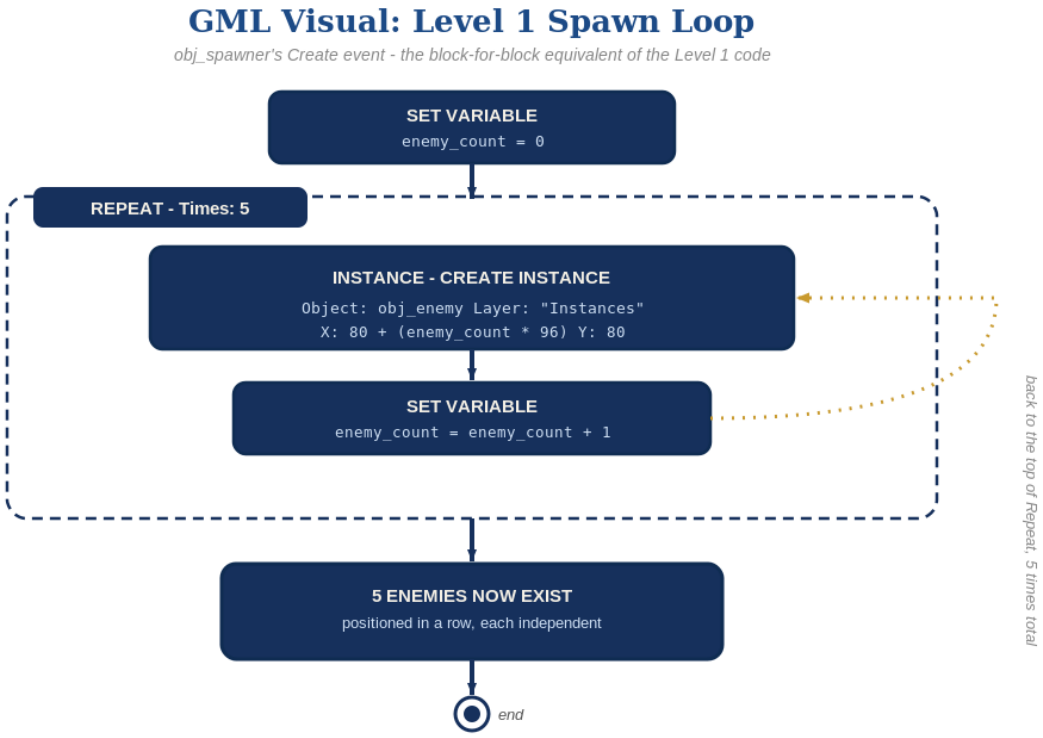
### Complete Level 3 solution (all 3 goals combined)

The spawner loop with the visual idea added - sound needs no changes since it already lives on obj_bullet:

```
// obj_spawner's Create event:
for (var enemy_count = 0; enemy_count < 5; enemy_count++) {
    var new_enemy = instance_create_layer(80 + enemy_count * 96, 80,
"Instances", obj_enemy);
    with (new_enemy) {
        if (enemy_count % 2 == 0) {
            image_blend = make_color_rgb(200, 25, 55);
        } else {
            image_blend = make_color_rgb(230, 100, 130);
        }
```

```
    }
}
```

## Alternate Path Notes: GML Visual




*Five blocks, three of them wired inside one Repeat - the exact same shape as the Activity Diagram above, just built with GML Visual's own blocks instead of typed code.*

*The five blocks above, wired together - the block-for-block equivalent of the Level 1 code.*

A "Set Variable" block (Variable = enemy_count, Value = 0) initializes the loop counter, a "Repeat" block (Times = 5) is the block equivalent of the for loop's condition, an "Instance - Create Instance" block inside it (Object = obj_enemy, Layer = "Instances", X = 80 + (enemy_count * 96), Y = 80) does the spawning, and a second "Set Variable" block (Variable = enemy_count, Value = enemy_count + 1) right after it increments the counter - together these five blocks are the exact equivalent of the code path's for (var enemy_count = 0; enemy_count < 5; enemy_count++) { instance_create_layer(...) }.

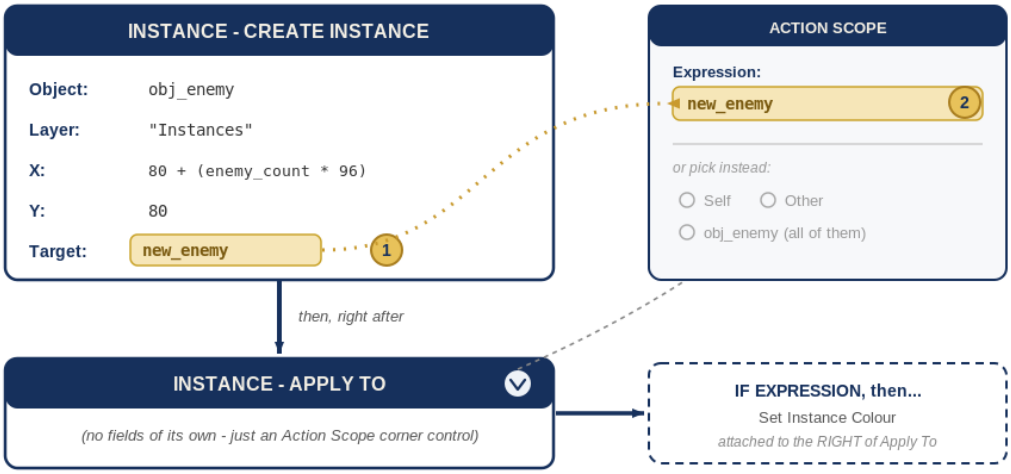
The Level 3 Visual idea is the one piece that isn't identical between Code and Visual, since it lives inside the spawn loop itself rather than on obj_enemy: give Create Instance a Target field set to a new local variable, new_enemy, to capture a reference to the instance it just made - the code path now captures it under this exact same name. Then drag an "Instance - Apply To" block right after it, and in Apply To's own Action Scope pop-up, type new_enemy into the Expression field at the top instead of picking Self/Other/an object - this re-points "Self" to that one captured instance for anything nested inside Apply To. Attach an If/Else block to the RIGHT of Apply To (not below it, which is what actually nests the If/Else inside its scope) on enemy_count mod 2 = 0, and inside each branch drop a "Set Instance Colour" block - Colour = make_color_rgb(200, 25, 55) in the If branch, make_color_rgb(230, 100, 130) in the Else branch, matching the code path's if/else exactly, block for statement.

This resolves a case that briefly looked like a genuine exception to preferring purpose-built blocks: Set Instance Colour's own Action Scope dropdown only lists Self, Other, or a specific object asset (which would recolor every existing enemy, not just the new one), so it looks like it can't reach an instance held in a variable - the same limit Set Instance Variable has. Apply To is what closes that gap: its Expression field accepts the exact same captured variable Create Instance's Target field produced, and nesting the purpose-built color block inside that scope makes "Self" mean the one instance that variable refers to. No generic block or dot-notation workaround needed after all - see this lesson's Book Notes entry for how this was found, correcting an earlier note.

Sound needs no changes either way, since it lives on obj_bullet's own Collision event, not the spawner's.

**GML Visual Wireframe: Capturing One Spawned Instance**

*inside the Repeat block, obj_spawner's Create event - Create Instance's Target, handed off to Apply To's Action Scope*

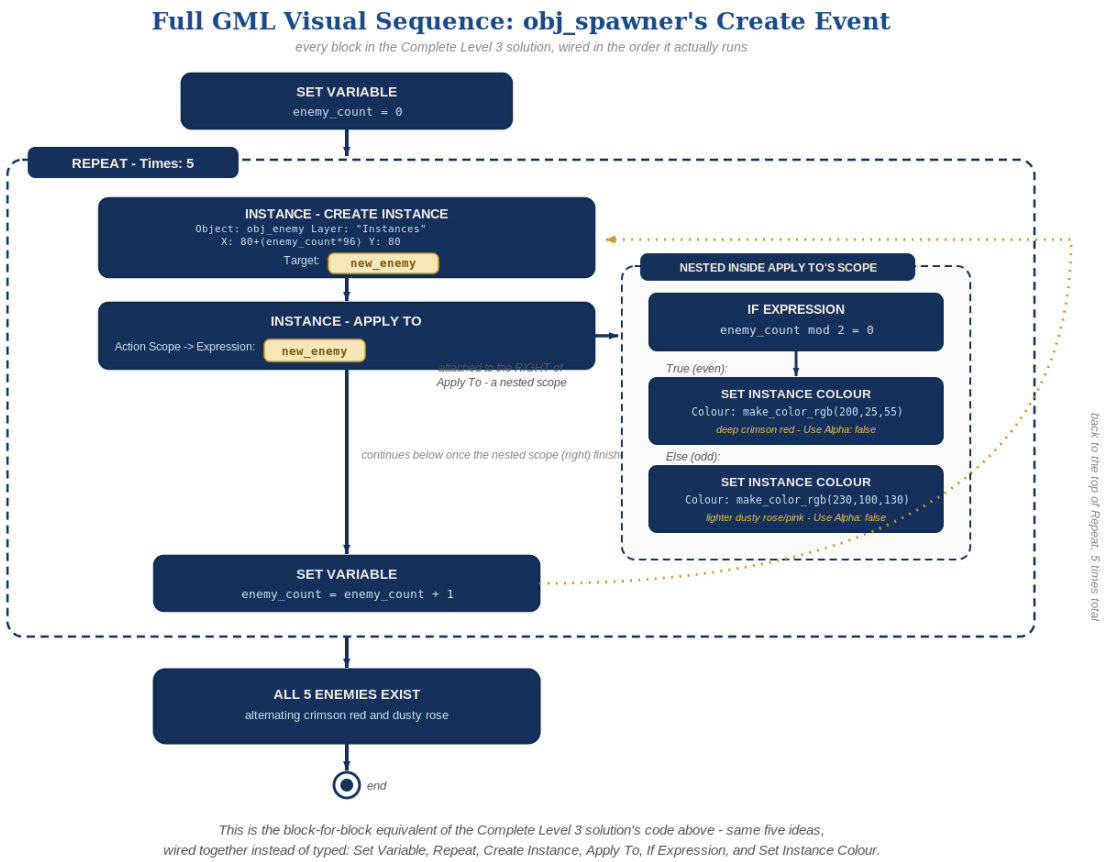



*1. Target is the only place Create Instance lets you capture a reference to the exact instance it just made.*
*2. Apply To's Action Scope Expression field is where that captured name goes instead of Self/Other/an object - everything attached to the RIGHT of Apply To (not below it) then runs with "Self" meaning that one instance, so the purpose-built Set Instance Colour block works normally - no generic block or typed workaround needed.*

*The exact block wiring behind that Target field - Create Instance's Target captures the new instance, and Apply To's Action Scope Expression field is what carries that capture into every block nested to its right, including the purpose-built Set Instance Colour block.*

### Complete Visual Sequence (all blocks combined)

Same idea as the Complete Level 3 solution shown in code above - here's every block in that solution, wired in the order it actually runs, not just the two blocks doing the tricky part.




*obj_spawner's entire Create event in GML Visual - Set Variable, Repeat, Create Instance, Apply To with its Expression-scoped If/Else color branch, and the counter increment, exactly mirroring the code path's for loop.*

- Keeping the same Structure principle (a spawner only spawns; behavior lives on the spawned object) whether a student is in Code or Visual reinforces that it's a design idea, not a syntax rule.
- Naming the exact block, category, and field for every step (rather than describing the shape of the logic) matters more in GML Visual than in most engines' visual tracks - and this lesson's own color-switch idea is a good example of why: the obvious-looking purpose-built block (Set Instance Colour) looks like it can't target a specific, dynamically-created instance at first glance, the same limit Set Instance Variable has. What actually unlocks it is a separate scope-setting block, Apply To, whose Action Scope Expression field takes the same captured variable Create Instance's Target produced - confirmed by hands-on testing, and worth remembering the next time a purpose-built block's dropdown looks like a dead end: check whether an Apply To block nested around it would solve it before reaching for a generic, typed-expression workaround.